

BIOTECHNOLOGY DATABASE PROJECT

Biobank and Biospecimen Management System

Development Environment: MySQL Workbench

Developed Using: MySQL Workbench

Student Name: Malak Mohamed Abouelwafa

Student ID: 231002183

Instructor: DR.Muhammad Elsayeh

Date: 15/8/2026

1.1 Problem Statement

Biobanks store and manage biological specimens that are collected from donors and used for biomedical and biotechnology research. Managing information about donors, consent, sample collection, storage, researchers, research projects, laboratory tests, and sample usage using separate or unstructured records can lead to duplicated information, inconsistent data, difficulty tracking samples, and errors in sample usage.

The purpose of this project is to design and implement a relational database system for a biobank and biospecimen management environment. The system provides a structured way to store and retrieve information about donors, biological samples, collection events, storage locations, aliquots, researchers, research projects, laboratory test requests, and sample usage.

The database is implemented using MySQL and developed in MySQL Workbench. The design demonstrates core relational database concepts including primary keys, foreign keys, integrity constraints, normalization, views, a trigger, a stored procedure, and SQL data operations.

1.2 Project Scope

The scope of the system is focused on managing the lifecycle of biological specimens within a research biobank. The system begins with registering donors and recording their consent. It then records sample collection events, sample types, biological samples, storage locations, and aliquots. The system also manages researchers and their participation in research projects. Researchers can request laboratory tests and record the use of samples for research activities.

The database is intended for research and biospecimen management purposes and does not implement clinical diagnosis, patient treatment, or direct clinical decision-making.

- Managing donor information.
- Recording donor consent.
- Managing sample collection events.
- Managing biological sample types.
- Tracking biological samples.
- Managing storage locations.
- Tracking sample aliquots.
- Managing researchers.
- Managing research projects.

- Assigning researchers to research projects.
- Managing laboratory test requests.
- Recording sample usage.

1.3 Assumptions

- Each donor is identified by a unique donor code.
- A donor may provide multiple biological samples through different collection events.
- Each collection event belongs to one donor.
- Each sample belongs to one collection event and one sample type.
- A sample may be divided into multiple aliquots.
- Each aliquot is stored in one storage location.
- Researchers may participate in multiple research projects.
- Each research project may have multiple researchers.
- The many-to-many relationship between researchers and research projects is represented using the researcher_project associative table.
- A researcher may submit multiple test requests.
- A biological sample may be associated with multiple test requests.
- Sample usage records identify the researcher, project, sample, date, and volume used.
- Donor information is represented using anonymized donor codes rather than real personal identifying information.

1.4 Major Business Rules

Rule	Business Rule
BR1	Each donor must have a unique donor code.
BR2	A donor can have zero or many collection events.
BR3	Each collection event must belong to exactly one donor.
BR4	Each biological sample must belong to one collection event and one sample type.
BR5	Each sample must have a unique sample code.
BR6	A sample can have multiple aliquots.
BR7	Each aliquot must be associated with one storage location.

BR8	A researcher can participate in multiple research projects, and a research project can have multiple researchers.
BR9	The researcher-project many-to-many relationship is resolved through the researcher_project associative table.
BR10	Sample usage volume must be greater than zero.
BR11	Sample usage must not exceed the remaining sample volume.
BR12	Foreign key constraints prevent records from referencing non-existing related records.

2. Entities, Attributes and Keys

The database contains 13 relational tables. Each table represents a meaningful entity or associative entity in the biobank system. Primary keys uniquely identify records, while foreign keys establish relationships between related entities.

Entity	Primary Key	Important Attributes	Foreign Keys	Purpose
Donor	donor_id	donor_code, date_of_birth, sex, blood_type, registration_date, status	None	Stores donor information.
Consent	consent_id	consent_date, consent_type, consent_status, expiry_date	donor_id	Stores donor consent information.
Sample Type	sample_type_id	type_name, description	None	Defines biological sample types.
Collection Event	collection_id	collection_date, collection_site, collector_name, notes	donor_id	Records specimen collection events.
Sample	sample_id	sample_code, initial_volume_ml, remaining_volume_ml, collection_date, status	collection_id, sample_type_id	Stores biological sample information.
Storage Location	location_id	freezer_code, rack_code, box_code,	None	Stores sample storage locations.

		position_code, temperature_c, capacity, status		
Aliquot	aliquot_id	aliquot_code, volume_ml, created_date, status	sample_id, location_id	Represents portions of biological samples.
Researcher	researcher_id	researcher_code, first_name, last_name, email, department, role	None	Stores researcher information.
Research Project	project_id	project_code, project_name, description, start_date, end_date, status	None	Stores research projects.
Researcher Project	researcher_id + project_id	assigned_date, project_role	researcher_id, project_id	Resolves the researcher-project many-to-many relationship.
Test Type	test_type_id	test_name, description, standard_unit	None	Defines laboratory test types.
Test Request	test_request_id	request_date, completion_date, result, status	sample_id, researcher_id, test_type_id	Records laboratory test requests.
Sample Usage	sample_usage_id	usage_date, volume_used_ml, purpose, notes	sample_id, researcher_id, project_id	Tracks use of samples in research.

3. Entity-Relationship Diagram

The ER diagram represents the conceptual structure of the Biobank and Biospecimen Management System. It identifies the main entities, their attributes, primary keys, foreign keys, and relationships. The model contains several one-to-many relationships, such as one donor having multiple collection events and one collection event producing multiple samples.

The model also contains a many-to-many relationship between researchers and research projects. This relationship is resolved through the researcher_project associative table, which stores assignment-specific attributes such as the assigned date and project role.

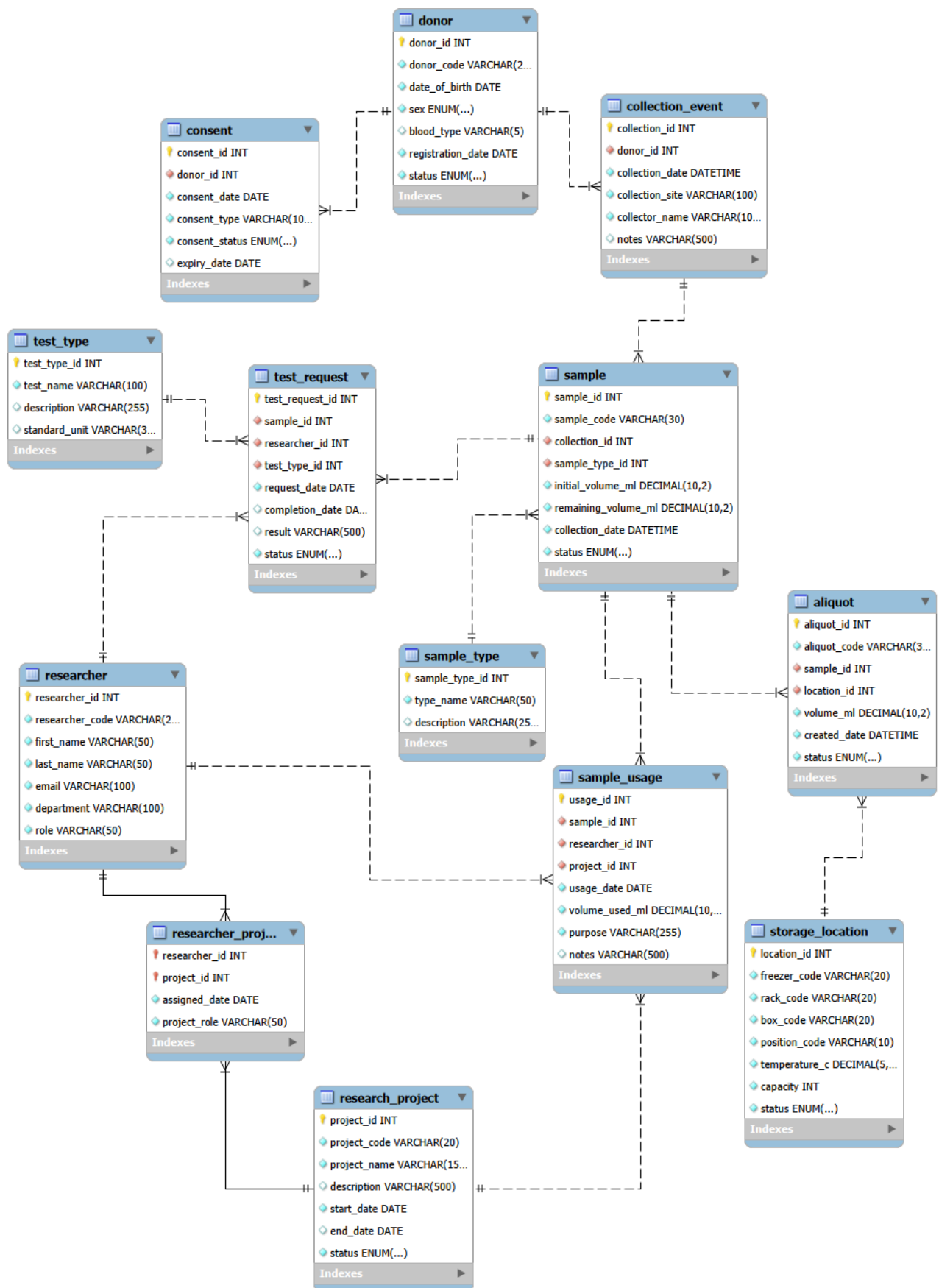


Figure 1. Entity-Relationship Diagram of the Biobank and Biospecimen Management System.

4. Relational Schema

The conceptual ER model was converted into a relational schema suitable for implementation in MySQL. Each strong entity was mapped to a relational table with a primary key. Foreign keys represent relationships between related tables. The many-to-many researcher-project relationship was transformed into the associative relation `researcher_project`.

```
DONOR(  
    donor_id PK,  
    donor_code UNIQUE,  
    date_of_birth,  
    sex,  
    blood_type,  
    registration_date,  
    status  
)
```

```
CONSENT(  
    consent_id PK,  
    donor_id FK,  
    consent_date,  
    consent_type,  
    consent_status,  
    expiry_date  
)
```

```
SAMPLE_TYPE(  
    sample_type_id PK,  
    type_name,  
    description  
)
```

```
COLLECTION_EVENT(  
    collection_id PK,  
    donor_id FK,  
    collection_date,  
    collection_site,  
    collector_name,  
    notes  
)
```

```
SAMPLE(  
    sample_id PK,  
    sample_code UNIQUE,  
    collection_id FK,  
    sample_type_id FK,  
    initial_volume_ml,  
    remaining_volume_ml,  
    collection_date,  
    status  
)
```

```
STORAGE_LOCATION(  
    location_id PK,  
    freezer_code,  
    rack_code,  
    box_code,  
    position_code,  
    temperature_c,  
    capacity,  
    status  
)
```

```
ALIQOT(  
    aliquot_id PK,  
    aliquot_code UNIQUE,  
    sample_id FK,  
    location_id FK,  
    volume_ml,  
    created_date,  
    status  
)
```

```
RESEARCHER(  
    researcher_id PK,  
    researcher_code UNIQUE,  
    first_name,  
    last_name,  
    email UNIQUE,  
    department,
```

```
    role
)
```

```
RESEARCH_PROJECT(
    project_id PK,
    project_code UNIQUE,
    project_name,
    description,
    start_date,
    end_date,
    status
)
```

```
RESEARCHER_PROJECT(
    researcher_id PK/FK,
    project_id PK/FK,
    assigned_date,
    project_role
)
```

```
TEST_TYPE(
    test_type_id PK,
    test_name UNIQUE,
    description,
    standard_unit
)
```

```
TEST_REQUEST(
    test_request_id PK,
    sample_id FK,
    researcher_id FK,
    test_type_id FK,
    request_date,
    completion_date,
    result,
    status
)
```

```
SAMPLE_USAGE(  
  sample_usage_id PK,  
  sample_id FK,  
  researcher_id FK,  
  project_id FK,  
  usage_date,  
  volume_used_ml,  
  purpose,  
  notes  
)
```

5. Normalization to Third Normal Form

5.1 First Normal Form (1NF)

The database satisfies First Normal Form because each table contains atomic attributes, meaning that each field stores a single value rather than a repeating group or multi-valued field. For example, researcher first name, last name, email, department, and role are stored as separate attributes.

5.2 Second Normal Form (2NF)

The database satisfies Second Normal Form because it is in First Normal Form and all non-key attributes depend on the complete primary key. This is especially relevant to researcher_project, whose identifying key is the combination of researcher_id and project_id. Attributes such as assigned_date and project_role describe the assignment and depend on the complete combination.

5.3 Third Normal Form (3NF)

The database satisfies Third Normal Form because non-key attributes depend only on the primary key and not on other non-key attributes. For example, sample type information is stored separately in sample_type instead of being repeated in every sample record. Researcher information is stored in researcher rather than being repeated in test requests or sample usage records.

The separation of donors, researchers, projects, sample types, test types, and storage locations reduces redundancy and helps prevent update, insertion, and deletion anomalies. Therefore, the final relational design is normalized to at least Third Normal Form (3NF).

6. Database Implementation

The database was implemented using MySQL and developed using MySQL Workbench. The implementation is organized into SQL scripts so that the database can be recreated and executed in a reproducible manner.

SQL File	Purpose
Biobank and Biospecimen Management System.sql	Creates the database tables, primary keys, foreign keys, constraints, and indexes.
load_data.sql	Loads meaningful test data into the database.
queries.sql	Contains retrieval, joins, aggregation, subqueries, INSERT, UPDATE, and DELETE operations.
views.sql	Creates the required database views.
triggers_procedures.sql	Creates the trigger and stored procedure used by the system.
testing.sql	Contains validation and testing queries, if maintained as a separate file.

6.1 Integrity Constraints

Primary keys uniquely identify rows in each table. Foreign keys enforce referential integrity between related entities. UNIQUE constraints are used for identifiers such as donor_code, sample_code, researcher_code, researcher_email, project_code, and test_name where applicable. NOT NULL constraints are used for attributes that are required for valid records. CHECK constraints and the sample-usage trigger are used where appropriate to enforce business rules.

6.2 Indexes

Indexes were used on selected attributes that are commonly searched or used in joins. Appropriate indexing helps MySQL locate records efficiently without changing the logical design of the database.

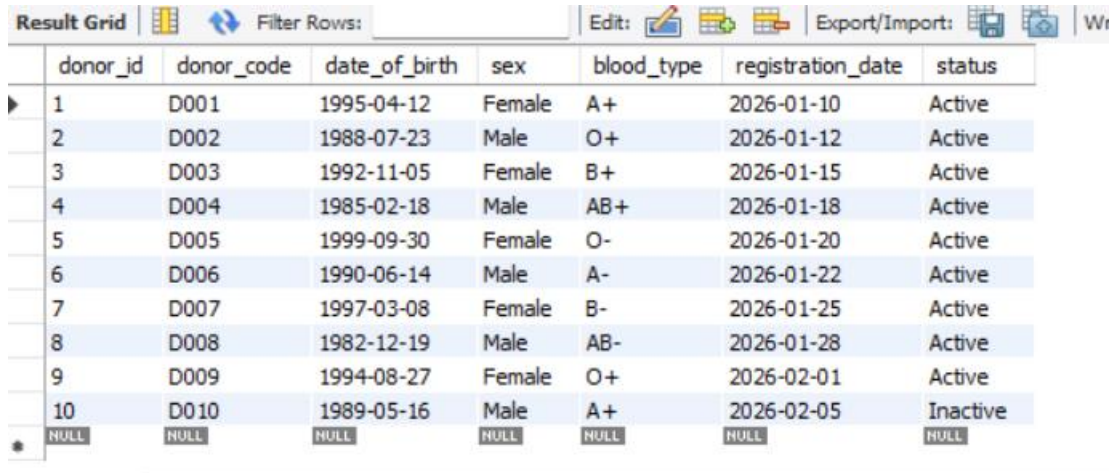
7. Sample Data and SQL Operations

Meaningful sample data was inserted into the database to demonstrate the functionality of the system. The records represent realistic biobank entities such as donors, biological samples, researchers, research projects, laboratory tests, and sample usage. Small lookup tables may contain fewer records when logically justified.

7.1 Data Retrieval

```
SELECT * FROM donor;
```

This query retrieves all donor records from the donor table.



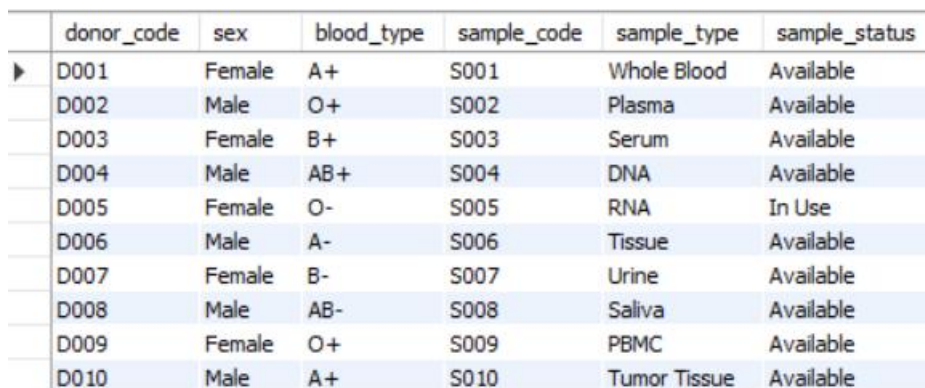
	donor_id	donor_code	date_of_birth	sex	blood_type	registration_date	status
▶	1	D001	1995-04-12	Female	A+	2026-01-10	Active
	2	D002	1988-07-23	Male	O+	2026-01-12	Active
	3	D003	1992-11-05	Female	B+	2026-01-15	Active
	4	D004	1985-02-18	Male	AB+	2026-01-18	Active
	5	D005	1999-09-30	Female	O-	2026-01-20	Active
	6	D006	1990-06-14	Male	A-	2026-01-22	Active
	7	D007	1997-03-08	Female	B-	2026-01-25	Active
	8	D008	1982-12-19	Male	AB-	2026-01-28	Active
	9	D009	1994-08-27	Female	O+	2026-02-01	Active
	10	D010	1989-05-16	Male	A+	2026-02-05	Inactive
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2. Donor records retrieved

7.2 Join Operations

```
SELECT d.donor_code, s.sample_code, s.status
FROM donor d
JOIN collection_event ce ON d.donor_id = ce.donor_id
JOIN sample s ON ce.collection_id = s.collection_id;
```

The join combines donor, collection event, and sample information to show which samples belong to which donors.



	donor_code	sex	blood_type	sample_code	sample_type	sample_status
▶	D001	Female	A+	S001	Whole Blood	Available
	D002	Male	O+	S002	Plasma	Available
	D003	Female	B+	S003	Serum	Available
	D004	Male	AB+	S004	DNA	Available
	D005	Female	O-	S005	RNA	In Use
	D006	Male	A-	S006	Tissue	Available
	D007	Female	B-	S007	Urine	Available
	D008	Male	AB-	S008	Saliva	Available
	D009	Female	O+	S009	PBMC	Available
	D010	Male	A+	S010	Tumor Tissue	Available

Figure 3. Joined donor and sample information.

7.3 Aggregation

```
SELECT d.donor_code, COUNT(s.sample_id) AS sample_count
FROM donor d
JOIN collection_event ce ON d.donor_id = ce.donor_id
JOIN sample s ON ce.collection_id = s.collection_id
GROUP BY d.donor_id, d.donor_code;
```

The aggregation query counts the number of samples associated with each donor.

	donor_code	number_of_samples
►	D001	1
	D002	1
	D003	1
	D004	1
	D005	1
	D006	1
	D007	1
	D008	1
	D009	1
	D010	1

Figure 4. Sample count per donor.

7.4 Subquery

```
SELECT sample_code, remaining_volume_ml
FROM sample
WHERE remaining_volume_ml >
      (SELECT AVG(remaining_volume_ml) FROM sample);
```

The subquery calculates the average remaining volume and returns samples whose remaining volume is above that average.

	sample_code	remaining_volume_ml
►	S001	18.00
	S003	15.00
	S006	20.00
	S007	27.00
	S010	18.00

Figure 5. Samples with above-average remaining volume.

7.5 INSERT, UPDATE and DELETE

The database supports data modification operations. INSERT is used to add new records, UPDATE changes existing records, and DELETE removes records when deletion is permitted by the defined relationships and constraints. Temporary test records can be used to demonstrate these operations without changing the permanent project dataset.

8. Views, Trigger and Stored Procedure

8.1 View 1 – donor_sample_view

The donor_sample_view provides a convenient combined representation of donor and sample information. It simplifies frequently required retrieval by storing a reusable SELECT definition containing the required joins.

```
SELECT * FROM donor_sample_view;
```

	donor_id	donor_code	sex	blood_type	donor_status	sample_id	sample_code	sample_type	initial_volum
▶	1	D001	Female	A+	Active	1	S001	Whole Blood	20.00
	2	D002	Male	O+	Active	2	S002	Plasma	15.00
	3	D003	Female	B+	Active	3	S003	Serum	18.00
	4	D004	Male	AB+	Active	4	S004	DNA	10.00
	5	D005	Female	O-	Active	5	S005	RNA	8.00
	6	D006	Male	A-	Active	6	S006	Tissue	25.00
	7	D007	Female	B-	Active	7	S007	Urine	30.00
	8	D008	Male	AB-	Active	8	S008	Saliva	12.00
	9	D009	Female	O+	Active	9	S009	PBMC	16.00
	10	D010	Male	A+	Inactive	10	S010	Tumor Tissue	22.00

Figure 7. donor_sample_view output.

8.2 View 2 – test_request_view

The test_request_view combines test request information with related samples, researchers, and test types. It provides a simplified representation for reviewing laboratory test requests and their results.

```
SELECT * FROM test_request_view;
```

	test_request_id	sample_code	researcher_name	department	test_name	standard_unit	requ
▶	1	S001	Mariam Ali	Molecular Biology	DNA Concentration	ng/uL	2026-
	2	S002	Ahmed Hassan	Genetics	RNA Concentration	ng/uL	2026-
	3	S003	Sara Mohamed	Biochemistry	PCR Analysis	Positive/Negative	2026-
	4	S004	Omar Khaled	Molecular Biology	ELISA	IU/mL	2026-
	5	S005	Mona Samir	Genomics	Protein Concentration	mg/mL	2026-
	6	S006	Youssef Nabil	Immunology	DNA Purity	Ratio	2026-
	7	S007	Nour Ibrahim	Biotechnology	RNA Purity	Ratio	2026-
	8	S008	Karim Mostafa	Genetics	Cell Viability	%	2026-
	9	S009	Laila Fathy	Pathology	Biomarker Assay	ng/mL	2026-
	10	S010	Hany Adel	Molecular Biology	Sequencing Quality	Score	2026-

Figure 8. test_request_view output.

8.3 Trigger – check_sample_usage_volume

The database includes the trigger check_sample_usage_volume. It is executed before a sample usage record is inserted and validates the requested usage volume against the available remaining sample volume. This protects the database from recording invalid or physically impossible sample usage.

SHOW TRIGGERS;					
	Trigger	Event	Table	Statement	Timing
▶	check_sample_usage_volume	INSERT	sample_usage	BEGIN DECLARE available_volume DECIMAL(...	BEFORE

Figure 9. Trigger

8.4 Stored Procedure – get_donor_samples

The stored procedure get_donor_samples receives a donor identifier and retrieves the biological samples associated with that donor. This provides reusable database logic for donor-specific sample retrieval.

CALL get_donor_samples(1);								
	donor_code	sex	blood_type	sample_code	sample_type	initial_volume_ml	remaining_volume_ml	sample_s
▶	D001	Female	A+	S001	Whole Blood	20.00	18.00	Available

Figure 10. Stored procedure execution.

9. Testing and Validation

Testing was performed in MySQL Workbench to verify that the database objects, relationships, SQL operations, and major business rules operate as expected. The results were inspected after executing the relevant SQL statements.

Test Case	Result
Donor retrieval	Donor records are displayed correctly
Join query	Related donor and sample information is returned
Aggregation	Sample counts are calculated correctly
Subquery	Records satisfying the subquery condition are returned
INSERT	A valid test record can be inserted
UPDATE	A test record can be modified
DELETE	A temporary test record can be removed
View 1	donor_sample_view returns expected data
View 2	test_request_view returns expected data
Trigger	Invalid sample usage is prevented
Stored procedure	Donor-specific samples are retrieved

10. Limitations and Conclusion

10.1 Limitations

The current system focuses on database management rather than providing a complete graphical user interface. Donor information is represented using anonymized identifiers and the database does not implement real patient-identifying information.

The system focuses on the core management of biospecimens, researchers, projects, testing, storage, and sample usage. Advanced features such as automated laboratory instrument integration, barcode scanning, user authentication, detailed audit logs, and a full application interface are outside the current scope.

10.2 Conclusion

The Biobank and Biospecimen Management System demonstrates the design and implementation of a relational biotechnology database using MySQL. The system contains 13 related tables and implements primary keys, foreign keys, integrity constraints, indexes, views, a trigger, and a stored procedure.

The database supports the management of donors, consent, collection events, biological samples, storage locations, aliquots, researchers, research projects, laboratory tests, and sample usage. The many-to-many relationship between researchers and research projects is resolved using an associative table.

The database is normalized to Third Normal Form to reduce redundancy and prevent common data anomalies. SQL operations including retrieval, joins, aggregation, subqueries, insertion, updating, and deletion were implemented. Overall, the project demonstrates the database development lifecycle from requirements analysis and conceptual design through relational implementation and testing.